## ¿Cuál es la diferencia entre una lista y una tupla en Python?

La principal diferencia entre ellas es que las listas son mutables y se definen con () y las tuplas son inmutables y se definen con []

Un ejemplo de lista seria:

```
lista = [1, 2, 3]
lista[0] = 0

print(lista)
```

Esto nos devolverá [0, 2, 3] cambiando la primera posición de la lista

Por el contrario, con la tupla:

```
tupla = (1, 2, 3)
tupla[0] = 0

print(tupla)
```

Esto nos devolverá un error ya que las tuplas no aceptan ese tipo de cambios.

Un método para agregar datos a una tupla seria:

```
tupla_nueva = tupla + (4,)

print(tupla_nueva)
```

La salida seria *(1, 2, 3, 4)*

## ¿Cuál es el orden de las operaciones?

El orden es el mismo que usan las matemáticas de toda la vida. Siguiendo este orden:

1. Paréntesis ()
2. Exponenciales $2^2$
3. Multiplicar * Dividir \
4. Sumar + Restar –

Primero resolvemos el interior de los paréntesis, después los exponenciales o potencias, seguido de multiplicaciones y divisiones y por último sumas y restas.

## ¿Qué es un diccionario Python?

Un diccionario es una forma de almacenar datos en pares de Clave\Valor. Se define con {} en ellas se almacena una clave y un valor por línea. La clave es algo único no se puede repetir ya sean letras o números, como un DNI no puede haber dos personas con el mismo numero. Y a su lado el valor asociado a esa llave. Por ejemplo:

```
persona = {
    "nombre": "Juan",
    "edad": 64,
    "ciudad": "Madrid"
}
```

En el ejemplo las claves serian la parte izquierda dentro de {} y su valor la parte derecha.

Su ejecutásemos print(persona["ciudad"]) nos devolvería Madrid ya que estaríamos pidiendo que dentro del diccionario persona nos de el valor de la clave ciudad.

## ¿Cuál es la diferencia entre el método ordenado y la función de ordenación?

Ambas sirven para para ordenar tienen sus diferencias. El método de ordenación ordena los datos de forma permanente, en cambio la función de ordenación te devuelve los datos ordenados sin modificar el orden principal, así podríamos llamar a los datos con el orden que se les dio originalmente. Por ejemplo:

```
metodo_ordenado = [5, 2, 9, 1]
metodo_ordenado.sort()

print(metodo_ordenado)


funcion_ordenacion = [5, 2, 9, 1]
función_de_ordenacion = sorted(funcion_ordenacion)

print(funcion_ordenacion)
print(función_de_ordenacion)
```

## ¿Qué es un operador de reasignación?

Nos permite actualizar el valor de una variable de forma abreviada y fácil.

Ejemplo: si tenemos un valor de 5 y quisiésemos sumar 5 lo primero que pensamos es en

```
x = 5
x = x + 5
```

Pero podemos abreviarlo con un operador de reasignación

```
x = 5
x += 5
```

Si llamamos a x nos devolverá 10

Hay varios operadores que podemos usar y todos se usan de esta misma forma, multiplicar, restar, Etc…

- Sumar +=
- Restar -=
- Multiplicar *=
- Dividir /=
- Potencia **=
- Resto división %=
- Division sin decimal //=